

Unidad Didáctica

**Programación Modular,
Unidad IV**

Índice de Contenido

Contenido

Introducción.....	3
Desarrollo de Contenidos	4
Introducción a la programación modular	4
Criterios de descomposición modular	5
Concepto de subprograma	7
Transferencia de información: Los parámetros	8
Lista de parámetros actuales y formales.....	10
Correspondencia de parámetros	12
Paso de parámetros por valor y por referencia	14
Variables locales y formales.....	16
Efectos laterales	17
Ámbito de un identificador	19
Reglas de ámbito.....	19
Acciones y funciones.....	21
Diferencia entre acciones y funciones	21
Ventajas de los subprogramas.....	22
Recursividad.....	24
Tipos de recursividad	26
Bibliografía y Webgrafía	28
Glosario	29

Introducción



¡Bienvenido al apasionante mundo de la programación modular! En este tema vamos a explorar los conceptos básicos de la programación modular y cómo esta técnica puede ayudarnos a construir programas más eficientes, sencillos y fáciles de mantener. En primer lugar, veremos los criterios de descomposición modular y cómo estos nos permiten dividir un programa en piezas más pequeñas y manejables. A continuación, profundizaremos en el concepto de subprograma, el cual es una pieza de código que puede ser invocada desde cualquier parte del programa.

Hablaremos sobre los parámetros, que son la manera en que un subprograma recibe información desde el programa principal, y cómo se manejan los valores pasados por referencia y por valor. También discutiremos sobre las variables locales y formales, y cómo se relacionan entre sí.

Posteriormente, nos adentraremos en la diferencia entre acciones y funciones, y veremos cómo cada una de estas puede ser utilizada para resolver diferentes problemas. Asimismo, hablaremos sobre las ventajas que nos brindan los subprogramas, como la reutilización de código y la facilidad para encontrar y corregir errores.

Por último, abordaremos la recursividad, que es la capacidad de un subprograma de invocarse a sí mismo. Exploraremos los diferentes tipos de recursividad y veremos cómo esta técnica puede ser utilizada para resolver problemas complejos de manera sencilla. ¡Prepárate para adentrarte en el fascinante mundo de la programación modular!

Introducción a la programación modular

La programación modular es una técnica de diseño de software que permite dividir un programa en partes más pequeñas y manejables llamadas módulos. Cada módulo es una unidad independiente que se encarga de realizar una tarea específica dentro del programa. Estos módulos pueden ser reutilizados en diferentes partes del programa o en otros programas.

La programación modular facilita la comprensión y el mantenimiento del código, ya que cada módulo es más fácil de entender y modificar que un programa completo. Además, permite que diferentes programadores trabajen en diferentes módulos de forma simultánea sin afectar al resto del programa.

Para crear un programa modular, es importante seguir ciertos principios de diseño. En primer lugar, cada módulo debe tener una funcionalidad clara y específica, y no debe depender de otros módulos para realizar su tarea. Además, se deben definir claramente las entradas y salidas de cada módulo, para asegurar que se comuniquen correctamente con el resto del programa.

Un ejemplo de programación modular puede ser el diseño de un sistema de gestión de una biblioteca. En este caso, se podrían crear módulos para la gestión de préstamos, la gestión de libros y la gestión de usuarios. Cada uno de estos módulos tendría su propia funcionalidad y se comunicaría con los demás a través de una interfaz definida.

Otro ejemplo podría ser el desarrollo de una aplicación de compras en línea. En este caso, se podrían crear módulos para la gestión de productos, la gestión de usuarios

y la gestión de pedidos. Cada uno de estos módulos tendría su propia funcionalidad y se comunicaría con los demás a través de una interfaz definida.

En conclusión, la programación modular es una técnica de diseño de software que permite dividir un programa en partes más pequeñas y manejables. Esto facilita la comprensión, el mantenimiento y la reutilización del código, lo que hace que sea una práctica esencial para el desarrollo de software de alta calidad.

Criterios de descomposición modular

La descomposición modular en Python es un proceso clave para el diseño de programas de alta calidad y fácilmente mantenibles. Para realizar una buena descomposición modular, es importante seguir algunos criterios que ayudarán a crear módulos más eficientes y fáciles de entender.

Uno de los criterios de descomposición modular en Python es la cohesión. La cohesión se refiere a la medida en que las tareas de un módulo están relacionadas entre sí. Un módulo altamente cohesivo debería contener funciones o métodos que se relacionen directamente con la funcionalidad del módulo. Por ejemplo, en un módulo de cálculo de impuestos, se deberían encontrar funciones para el cálculo de impuestos, y no para la gestión de cuentas de usuario.

Otro criterio importante es el acoplamiento. El acoplamiento se refiere a la relación entre módulos. Un módulo con un acoplamiento bajo debería tener una independencia adecuada de los otros módulos del programa. Es decir, no debería depender de otros módulos para realizar sus tareas. En caso de ser necesario, el módulo debería recibir los datos de entrada necesarios para su correcto funcionamiento a través de una interfaz clara y definida.

Un tercer criterio de descomposición modular es la simplicidad. Los módulos en Python deben ser lo más simples y fáciles de entender posible. Esto implica que cada módulo debe realizar tareas específicas y claras, sin incluir complejidades adicionales o innecesarias. Los nombres de las funciones o métodos deben ser claros y descriptivos, lo que ayudará a la comprensión del código.

Finalmente, otro criterio importante de descomposición modular en Python es la reutilización. Un módulo bien diseñado debería poder ser reutilizado en diferentes partes del programa, y en otros programas. Esto significa que los módulos deben ser lo más genéricos posible, y no incluir referencias específicas a otras partes del programa o del entorno.

Un ejemplo de descomposición modular en Python puede ser el desarrollo de una aplicación de gestión de tareas. En este caso, se podrían crear módulos para la gestión de tareas, la gestión de proyectos, la gestión de usuarios y la gestión de permisos. Cada uno de estos módulos debería ser coherente con su funcionalidad específica, tener un bajo acoplamiento con otros módulos, ser simple y claro en su diseño y tener la capacidad de ser reutilizado en otros programas.

En conclusión, seguir los criterios de descomposición modular en Python es esencial para el diseño de programas de alta calidad y fácilmente mantenibles. La cohesión, el acoplamiento, la simplicidad y la reutilización son los principios fundamentales para la creación de módulos eficientes y fáciles de entender.

Concepto de subprograma

En Python, un subprograma es una sección de código que realiza una tarea específica y puede ser llamada desde cualquier parte del programa principal. También se le conoce como función o procedimiento, y su uso permite la modularización del código, mejorando la legibilidad y la eficiencia del programa.

Un subprograma se define mediante la palabra clave "def", seguida del nombre del subprograma y los parámetros que recibe, si los tiene. Estos parámetros son los valores de entrada que se utilizan dentro del subprograma para realizar las operaciones necesarias. También se pueden definir variables dentro del subprograma que solo se utilizan dentro de él.

Una vez que se ha definido el subprograma, puede ser llamado desde cualquier parte del programa principal utilizando su nombre. Al llamar al subprograma, se le pasan los valores de entrada necesarios y el subprograma realiza su tarea específica y devuelve un resultado, si es necesario.

Un ejemplo sencillo de subprograma en Python puede ser el siguiente:

```
def suma(a, b):  
    c = a + b  
    return c
```

En este ejemplo, se ha definido un subprograma llamado "suma", que recibe dos parámetros "a" y "b". Dentro del subprograma, se realiza la suma de estos dos valores y se guarda el resultado en la variable "c". Finalmente, se devuelve el valor de "c". Este subprograma puede ser llamado desde cualquier parte del programa principal de la siguiente manera:

```
resultado = suma(2, 3)
```

En este caso, se está llamando al subprograma "suma" y se le están pasando los valores 2 y 3 como parámetros. El resultado de la suma, que es 5, se guarda en la variable "resultado".

En conclusión, los subprogramas son una herramienta fundamental en Python y en otros lenguajes de programación, ya que permiten la modularización del código, mejorando su legibilidad y eficiencia. Su uso es esencial en programas grandes y complejos, y su correcta implementación puede ayudar a evitar errores y a simplificar el proceso de depuración.

Transferencia de información: Los parámetros

En Python, los parámetros son valores que se pasan a una función o subprograma para que éste pueda realizar una tarea específica. Los parámetros permiten la transferencia de información entre el programa principal y los subprogramas, lo que los hace fundamentales en la programación modular.

Existen dos tipos de parámetros en Python: los parámetros posicionales y los parámetros de palabra clave. Los parámetros posicionales son aquellos que se pasan a una función en el orden en que están definidos en la misma. Los parámetros de palabra clave, por su parte, se pasan a una función utilizando el nombre del parámetro, lo que permite cambiar el orden en el que se pasan los parámetros sin afectar el resultado.

A continuación, se presenta un ejemplo de una función que utiliza ambos tipos de parámetros:


```
def suma(a, b, c=0):  
    return a + b + c
```

En este ejemplo, se ha definido una función llamada "suma" que recibe tres parámetros: "a", "b" y "c". El parámetro "c" tiene un valor predeterminado de cero, lo que significa que si no se le pasa un valor en la llamada a la función, su valor será cero.

Al llamar a esta función, se pueden utilizar ambos tipos de parámetros. Por ejemplo:

```
resultado1 = suma(2, 3)  
resultado2 = suma(2, 3, 4)  
resultado3 = suma(c=2, b=3, a=1)
```

En el primer caso, se están pasando únicamente los parámetros "a" y "b" a la función "suma". En el segundo caso, se están pasando los tres parámetros, por lo que se está utilizando el valor predeterminado de "c". En el tercer caso, se están pasando los parámetros utilizando los nombres de los mismos, lo que permite cambiar el orden en que se pasan sin afectar el resultado.

Los parámetros son una herramienta fundamental en Python y en otros lenguajes de programación, ya que permiten la transferencia de información entre el programa principal y los subprogramas. Su correcta utilización puede ayudar a simplificar el código y a evitar errores.

Lista de parámetros actuales y formales

En Python, es común que una función reciba una lista de parámetros en su definición, lo que se conoce como "parámetros formales". Estos parámetros son variables que se utilizan dentro de la función para realizar una tarea específica.

Cuando se llama a una función en Python, se pueden pasar una serie de valores como argumentos, lo que se conoce como "parámetros actuales". Estos valores se asignan a los parámetros formales de la función en el orden en que se pasan.

Es importante tener en cuenta que la cantidad de parámetros actuales que se pasen a una función debe coincidir con la cantidad de parámetros formales que se hayan definido en la misma. De lo contrario, Python lanzará un error.

A continuación, se presenta un ejemplo que ilustra el uso de los parámetros formales y actuales en Python:

```
def suma(a, b):  
    return a + b
```

```
resultado = suma(2, 3)
```

En este ejemplo, se ha definido una función llamada "suma" que recibe dos parámetros formales: "a" y "b". En la llamada a la función, se están pasando dos parámetros actuales: el valor 2 y el valor 3. Estos valores se asignarán a los parámetros formales "a" y "b", respectivamente. La función "suma" devolverá la suma de ambos valores, que se asignará a la variable "resultado".

Es importante destacar que en Python, es posible utilizar una lista de parámetros variables en una función, lo que se conoce como "parámetros arbitrarios". En este caso, se utiliza el operador de asterisco (*) antes del nombre del parámetro formal para indicar que se trata de una lista variable de parámetros.

A continuación, se presenta un ejemplo de una función que utiliza parámetros arbitrarios:

```
def suma_arbitraria(*numeros):  
    resultado = 0  
    for numero in numeros:  
        resultado += numero  
    return resultado
```

```
resultado = suma_arbitraria(2, 3, 4, 5)
```

En este ejemplo, se ha definido una función llamada "suma_arbitraria" que utiliza parámetros arbitrarios. Se está utilizando el operador de asterisco (*) antes del nombre del parámetro formal "numeros", lo que indica que se trata de una lista variable de parámetros.

En la llamada a la función, se están pasando cuatro parámetros actuales: los valores 2, 3, 4 y 5. Estos valores se asignarán a la lista "numeros". La función recorrerá la lista y devolverá la suma de todos los valores, que se asignará a la variable "resultado".

En resumen, en Python, los parámetros formales son variables que se utilizan dentro de una función para realizar una tarea específica, mientras que los parámetros

actuales son valores que se pasan a la función cuando se la llama. Es importante asegurarse de que la cantidad de parámetros actuales coincida con la cantidad de parámetros formales definidos en la función. Además, es posible utilizar parámetros arbitrarios en una función utilizando el operador de asterisco (*).

Correspondencia de parámetros

En Python, la correspondencia de parámetros se refiere al proceso de asignar los valores de los parámetros actuales a los parámetros formales en una función. Es importante comprender cómo funciona la correspondencia de parámetros, ya que puede afectar el comportamiento y los resultados de una función.

Python utiliza dos métodos diferentes para la correspondencia de parámetros: la correspondencia por posición y la correspondencia por nombre.

La correspondencia por posición es el método más común y se basa en el orden de los parámetros. Los valores de los parámetros actuales se asignan a los parámetros formales en el mismo orden en que se definen. Por ejemplo:

```
def imprimir_datos(nombre, edad):
```

```
    print("Nombre:", nombre)
```

```
    print("Edad:", edad)
```

```
imprimir_datos("Juan", 25)
```

En este ejemplo, se definen dos parámetros formales: "nombre" y "edad". En la llamada a la función, se pasan dos parámetros actuales: "Juan" y 25. Python asigna el valor "Juan" al parámetro formal "nombre" y el valor 25 al parámetro formal "edad".

La correspondencia por nombre, por otro lado, se basa en los nombres de los parámetros formales en lugar de su posición. En este caso, los valores de los parámetros actuales se asignan a los parámetros formales según su nombre. Por ejemplo:

```
def imprimir_datos(nombre, edad):  
    print("Nombre:", nombre)  
    print("Edad:", edad)  
  
imprimir_datos(edad=25, nombre="Juan")
```

En este ejemplo, se definen dos parámetros formales: "nombre" y "edad". En la llamada a la función, se pasan dos parámetros actuales: "Juan" y 25, pero en un orden diferente al de la definición de la función. Sin embargo, como se utiliza la correspondencia por nombre, Python asigna el valor "Juan" al parámetro formal "nombre" y el valor 25 al parámetro formal "edad".

Es importante tener en cuenta que se pueden combinar la correspondencia por posición y por nombre en una misma llamada a una función. Sin embargo, si se utiliza la correspondencia por nombre, todos los parámetros formales deben ser nombrados en la llamada.

La correspondencia de parámetros es una parte fundamental de la programación en Python, ya que permite que las funciones se comuniquen con el resto del programa. Comprender cómo funciona la correspondencia de parámetros es esencial para crear funciones efectivas y fáciles de usar.

Paso de parámetros por valor y por referencia

En Python, los parámetros se pueden pasar a una función de dos maneras: por valor y por referencia. Es importante comprender la diferencia entre estos dos métodos, ya que pueden afectar la forma en que los valores se manipulan dentro de la función.

El paso de parámetros por valor se refiere a la copia del valor de un parámetro en una variable temporal dentro de la función. Es decir, se crea una copia del valor original y se trabaja con ella dentro de la función. Cualquier cambio realizado en la variable dentro de la función no afectará el valor original fuera de la función. Esto se debe a que la variable original fuera de la función está en una ubicación de memoria diferente a la variable temporal dentro de la función. Por ejemplo:

```
def sumar(numero):  
    numero = numero + 1  
    print("Valor dentro de la función:", numero)  
  
numero_original = 5  
sumar(numero_original)  
print("Valor fuera de la función:", numero_original)
```

En este ejemplo, se define una función "sumar" que toma un parámetro "numero" y le suma 1. En la llamada a la función, se pasa la variable "numero_original" con el valor 5. Dentro de la función, se crea una copia de "numero_original" llamada "numero" y se le suma 1. La variable "numero_original" fuera de la función no se ve afectada y sigue teniendo el valor 5.

El paso de parámetros por referencia, por otro lado, se refiere a pasar la ubicación de memoria de un objeto como parámetro. En este caso, cualquier cambio realizado

en el objeto dentro de la función se reflejará en el objeto original fuera de la función, ya que ambas variables apuntan a la misma ubicación de memoria. Por ejemplo:

```
def agregar_elemento(lista, elemento):  
    lista.append(elemento)  
    print("Valor dentro de la función:", lista)  
  
lista_original = [1, 2, 3]  
agregar_elemento(lista_original, 4)  
print("Valor fuera de la función:", lista_original)
```

En este ejemplo, se define una función "agregar_elemento" que toma dos parámetros: "lista" y "elemento". La función agrega el elemento a la lista y luego imprime la lista actualizada. En la llamada a la función, se pasa la variable "lista_original" que contiene la lista [1, 2, 3]. Dentro de la función, se agrega el elemento 4 a la lista y se imprime la lista actualizada. La variable "lista_original" fuera de la función ahora también contiene la lista actualizada [1, 2, 3, 4].

Es importante tener en cuenta que en Python, los objetos mutables (como las listas y los diccionarios) se pasan por referencia, mientras que los objetos inmutables (como las cadenas y los números) se pasan por valor. Comprender cómo funcionan el paso de parámetros por valor y por referencia es fundamental para evitar errores y crear funciones efectivas y fáciles de usar en Python.

Variables locales y formales

En Python, las variables pueden ser locales o formales. Es importante comprender la diferencia entre estas dos variables, ya que afectan la forma en que se manipulan los valores dentro de una función.

Una variable local es una variable definida dentro de una función y solo es accesible dentro de esa función. Es decir, su alcance está limitado a la función en la que se define y no se puede acceder desde fuera de la función. Por ejemplo:

```
def sumar(numero1, numero2):  
    resultado = numero1 + numero2  
    return resultado
```

```
print(sumar(2, 3))
```

En este ejemplo, se define una función "sumar" que toma dos parámetros "numero1" y "numero2". Dentro de la función, se define una variable local "resultado" que es la suma de "numero1" y "numero2". Luego, se devuelve el valor de "resultado". La variable "resultado" solo es accesible dentro de la función "sumar" y no se puede acceder desde fuera de ella.

Una variable formal, por otro lado, es un parámetro que se define en una función y se utiliza para recibir valores desde la llamada a la función. Es decir, su alcance está limitado a la función en la que se define, pero su valor es definido por la llamada a la función. Por ejemplo:


```
def restar(numero1, numero2):  
    resultado = numero1 - numero2  
    return resultado
```

```
print(restar(5, 2))
```

En este ejemplo, se define una función "restar" que toma dos parámetros "numero1" y "numero2". Estos parámetros son variables formales que se utilizan para recibir valores desde la llamada a la función. Dentro de la función, se define una variable local "resultado" que es la resta de "numero1" y "numero2". Luego, se devuelve el valor de "resultado". Los parámetros "numero1" y "numero2" son variables formales que se definen en la llamada a la función "restar(5, 2)".

Es importante tener en cuenta que, aunque una variable local y una variable formal pueden tener el mismo nombre, son dos variables diferentes y no están relacionadas entre sí. Además, el alcance de una variable local o formal está limitado a la función en la que se define. Entender la diferencia entre una variable local y una variable formal es fundamental para crear funciones efectivas en Python y evitar errores.

Efectos laterales

En programación, los efectos laterales se refieren a cualquier cambio en el estado del programa que no sea el valor devuelto por una función. En Python, los efectos laterales pueden ser sutiles y pueden conducir a errores difíciles de depurar.

Un ejemplo de efecto lateral en Python es cuando una función modifica una variable global. Las variables globales son aquellas que se definen fuera de una función y se pueden acceder desde cualquier parte del programa. Si una función modifica una variable global, está teniendo un efecto lateral en el estado del programa que no está relacionado con el valor devuelto por la función.

Por ejemplo:

```
contador = 0
```

```
def incrementar_contador():
```

```
    global contador
```

```
    contador += 1
```

```
incrementar_contador()
```

```
print(contador)
```

En este ejemplo, se define una variable global "contador" y una función "incrementar_contador" que modifica el valor de "contador" al agregarle 1. Dentro de la función, se utiliza la palabra clave "global" para indicar que se está modificando una variable global en lugar de una variable local. Luego, se llama a la función "incrementar_contador" y se imprime el valor de "contador". En este caso, la función "incrementar_contador" tiene un efecto lateral al modificar una variable global.

Los efectos laterales pueden ser útiles en algunos casos, pero también pueden ser una fuente de errores y hacer que el código sea más difícil de entender y depurar. Es una buena práctica evitar los efectos laterales siempre que sea posible y asegurarse de que las funciones sean puras, es decir, que solo dependan de los parámetros de entrada y no tengan efectos laterales en el estado del programa.

En resumen, los efectos laterales son cualquier cambio en el estado del programa que no sea el valor devuelto por una función. En Python, un ejemplo de efecto lateral es cuando una función modifica una variable global. Es importante tener en cuenta los efectos laterales al diseñar y depurar programas para evitar errores y hacer que el código sea más fácil de entender.

Ámbito de un identificador

Reglas de ámbito

Las reglas de ámbito en Python se refieren a las reglas que determinan dónde se pueden acceder y modificar variables en el código. En Python, el ámbito de una variable se determina por su ubicación dentro del código y puede ser global o local.

En primer lugar, las variables globales se definen fuera de cualquier función o clase y se pueden acceder desde cualquier parte del código. Estas variables pueden ser modificadas en cualquier parte del programa, incluyendo dentro de funciones o clases.

Por otro lado, las variables locales se definen dentro de una función o método y solo se pueden acceder desde dentro de esa función o método. Estas variables no existen fuera del ámbito en el que se definen.

Cuando se llama a una función, Python busca primero las variables dentro de la función antes de buscar las variables globales. Si se encuentra una variable local con el mismo nombre que una variable global, la variable local tiene prioridad sobre la variable global.

Además, es importante tener en cuenta que las variables locales solo existen durante la ejecución de la función. Una vez que la función se completa, todas las variables locales se eliminan de la memoria.

Veamos algunos ejemplos para ilustrar estas reglas de ámbito en Python:

```
variable_global = 10
```

```
def funcion_ejemplo():  
    variable_local = 20  
    print("Variable local dentro de la función:", variable_local)  
    print("Variable global dentro de la función:", variable_global)
```

```
funcion_ejemplo()  
print("Variable global fuera de la función:", variable_global)
```

En este ejemplo, se define una variable global llamada "variable_global" con un valor de 10 y una función llamada "funcion_ejemplo". Dentro de la función, se define una variable local llamada "variable_local" con un valor de 20. Luego, se imprimen ambos valores dentro de la función y se llama a la función. Finalmente, se imprime el valor de la variable global fuera de la función.

La salida del programa sería:

```
Variable local dentro de la función: 20  
Variable global dentro de la función: 10  
Variable global fuera de la función: 10
```

Como se puede ver, la variable local "variable_local" solo existe dentro de la función y no se puede acceder fuera de ella. La variable global "variable_global" se puede acceder tanto dentro como fuera de la función.

En resumen, las reglas de ámbito en Python determinan dónde se pueden acceder y modificar las variables en el código. Las variables globales se definen fuera de

cualquier función y se pueden acceder desde cualquier parte del código, mientras que las variables locales solo existen dentro de la función en la que se definen. Al llamar a una función, Python busca primero las variables locales antes de buscar las variables globales. Es importante tener en cuenta estas reglas de ámbito al diseñar y depurar programas para evitar errores y hacer que el código sea más fácil de entender.

Acciones y funciones

Diferencia entre acciones y funciones

En el ámbito de la programación, es común hablar de "acciones" y "funciones" para referirse a dos tipos de estructuras de código que cumplen distintas funciones. En Python, estas estructuras son muy utilizadas y es importante conocer la diferencia entre ambas.

Una acción en Python es una porción de código que realiza una tarea específica sin devolver ningún valor. Por ejemplo, una acción puede imprimir un mensaje en la consola, mover un objeto gráfico, o modificar una variable global. Las acciones se utilizan para realizar tareas que no requieren la devolución de resultados y que pueden ser invocadas varias veces desde diferentes partes del programa.

Por otro lado, una función en Python es una estructura de código que realiza una tarea específica y devuelve un valor o conjunto de valores como resultado. Las funciones pueden recibir uno o varios argumentos como entrada para realizar su tarea, y su principal objetivo es reutilizar código y simplificar el diseño de programas. Una función en Python puede ser invocada desde diferentes partes del programa y con diferentes argumentos, y siempre devolverá un resultado predecible.

La diferencia principal entre acciones y funciones en Python es que las acciones no devuelven ningún valor, mientras que las funciones sí lo hacen. Por ejemplo, si

tenemos una función que calcula el área de un círculo a partir de su radio, esta función devolverá el valor calculado como resultado. Por otro lado, si tenemos una acción que imprime un mensaje en la consola, esta acción no devolverá ningún valor.

Otra diferencia importante es que las funciones se utilizan para reutilizar código, mientras que las acciones son útiles para realizar tareas específicas sin necesidad de devolver resultados. En resumen, las acciones y las funciones son dos estructuras de código importantes en Python, y su uso adecuado puede mejorar la eficiencia y la calidad del código. Es importante conocer la diferencia entre ambas para elegir la estructura adecuada en cada situación.

Ventajas de los subprogramas

Los subprogramas en Python son bloques de código que se pueden llamar desde otras partes del programa para realizar una tarea específica. Los subprogramas, también conocidos como funciones o procedimientos, tienen varias ventajas que los hacen una herramienta importante en la programación.

Una de las principales ventajas de los subprogramas es la reutilización de código. Los subprogramas se pueden llamar desde diferentes partes del programa, lo que permite utilizar el mismo código para realizar tareas similares o idénticas en diferentes lugares. En lugar de escribir el mismo código repetidamente, se puede definir un subprograma una sola vez y llamarlo desde diferentes partes del programa, lo que reduce la cantidad de código que se debe escribir y mantener.

Otra ventaja de los subprogramas es que mejoran la legibilidad y la estructura del código. Al utilizar subprogramas, se puede separar la funcionalidad del código en piezas más pequeñas y manejables, lo que hace que sea más fácil entender cómo funciona el programa y cómo se relacionan las diferentes partes. Además, los

subprogramas pueden tener nombres descriptivos que reflejen su función, lo que facilita la comprensión del código.

Los subprogramas también permiten el encapsulamiento del código y la ocultación de detalles de implementación. Al definir un subprograma, se puede ocultar la forma en que se realiza una tarea específica y simplemente proporcionar una interfaz clara para llamarlo. Esto puede hacer que el programa sea más robusto y más fácil de mantener, ya que los cambios en la implementación del subprograma no afectarán a otras partes del programa que lo utilizan.

Por último, los subprogramas pueden mejorar la eficiencia del programa al permitir la modularización del código. Al dividir el programa en subprogramas más pequeños y manejables, se puede optimizar el rendimiento de cada subprograma y reducir la complejidad global del programa. Esto puede hacer que el programa sea más rápido y más fácil de depurar.

En conclusión, los subprogramas en Python son una herramienta importante en la programación ya que permiten la reutilización de código, mejoran la legibilidad y estructura del código, permiten el encapsulamiento del código y la ocultación de detalles de implementación, y pueden mejorar la eficiencia del programa. Su uso adecuado puede mejorar la calidad del código y facilitar el mantenimiento y la escalabilidad del programa.

Recursividad

La recursividad es una técnica en programación que permite a una función llamarse a sí misma para resolver un problema de manera iterativa. En Python, esta técnica se utiliza con frecuencia para resolver problemas que pueden ser expresados en términos de una serie de pasos similares.

Para implementar una función recursiva en Python, es importante establecer un caso base o condición de salida, que evite que la función se llame a sí misma indefinidamente. Además, es necesario identificar el caso recursivo, que es aquel en el que la función se llama a sí misma para resolver el problema.

Una de las ventajas de la recursividad es su capacidad para simplificar el código y hacerlo más fácil de leer y entender. Además, puede ser una solución elegante para algunos problemas que son difíciles de resolver de manera iterativa.

A continuación, se presenta un ejemplo de una función recursiva en Python que calcula el factorial de un número:

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factorial(n-1)
```

En este ejemplo, la función factorial se llama a sí misma para calcular el factorial del número n. El caso base es cuando n es igual a 1, en cuyo caso la función devuelve 1. El caso recursivo es cuando n es mayor que 1, en cuyo caso la función se llama a sí misma con el argumento n-1 y multiplica el resultado por n.

Otro ejemplo de recursividad en Python es la función para calcular el número de Fibonacci:

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)
```

En este ejemplo, la función fibonacci se llama a sí misma para calcular el número de Fibonacci para el valor n. El caso base es cuando n es igual a 0 o 1, en cuyo caso la función devuelve el valor correspondiente. El caso recursivo es cuando n es mayor que 1, en cuyo caso la función se llama a sí misma dos veces con los argumentos n-1 y n-2, y suma los resultados.

En resumen, la recursividad es una técnica poderosa en Python que permite a las funciones llamarse a sí mismas para resolver problemas de manera iterativa. Con la implementación adecuada del caso base y del caso recursivo, la recursividad puede ser una solución elegante y fácil de entender para algunos problemas.

Tipos de recursividad

La recursividad es una técnica de programación en la que una función se llama a sí misma para resolver un problema. En Python, al igual que en otros lenguajes de programación, existen diferentes tipos de recursividad que se pueden utilizar según el problema que se quiera resolver. A continuación, se explicarán los principales tipos de recursividad en Python:

1. Recursividad directa: en este tipo de recursividad, la función se llama a sí misma directamente. Es el tipo de recursividad más común y se utiliza para resolver problemas que se pueden dividir en subproblemas similares. Por ejemplo, la función factorial se puede definir de manera recursiva:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

En este ejemplo, la función factorial se llama a sí misma para calcular la factorial de un número menor. Cuando se alcanza el caso base ($n = 0$), la recursividad se detiene y se devuelve el resultado.

2. Recursividad indirecta: en este tipo de recursividad, una función llama a otra función que, a su vez, llama a la primera función. Este tipo de recursividad se utiliza cuando el problema se puede dividir en subproblemas que se resuelven con funciones diferentes pero relacionadas. Por ejemplo, la función gcd (máximo común divisor) se puede definir de manera recursiva utilizando la función mod (módulo):

```
def gcd(a, b):  
    if b == 0:  
        return a  
    else:  
        return gcd(b, a % b)
```

En este ejemplo, la función gcd se llama a sí misma indirectamente a través de la función mod.

3. Recursividad mutua: en este tipo de recursividad, dos o más funciones se llaman entre sí para resolver un problema. Este tipo de recursividad se utiliza cuando el problema se puede dividir en subproblemas que se resuelven con funciones diferentes pero que necesitan comunicarse entre sí. Por ejemplo, la función is_even (es par) y is_odd (es impar) se pueden definir de manera recursiva de la siguiente manera:

```
def is_even(n):  
    if n == 0:  
        return True  
    else:  
        return is_odd(n-1)  
  
def is_odd(n):  
    if n == 0:  
        return False  
    else:  
        return is_even(n-1)
```

En este ejemplo, la función `is_even` se llama a sí misma indirectamente a través de la función `is_odd`, y viceversa.

En resumen, la recursividad es una técnica poderosa para resolver problemas complejos y se puede aplicar de diferentes maneras según el problema que se quiera resolver. La recursividad directa es la más común, pero también existen otros tipos como la recursividad indirecta y la recursividad mutua. Es importante tener en cuenta que el uso excesivo de la recursividad puede llevar a problemas de rendimiento y a errores de desbordamiento de pila.

Bibliografía y Webgrafía

Sánchez, M. A. (2014). Fundamentos de programación. Alfaomega.

López, J. (2020). Programación en Python 3: Algoritmos, Programas y Aplicaciones. Alfaomega.

Acción: En programación, una acción se refiere a una tarea específica que realiza una parte del código, pero que no devuelve un valor. En el texto, se menciona una acción que imprime un mensaje en la consola.

Acciones y funciones: En Python, las acciones y las funciones son dos estructuras de código diferentes. Una acción es una porción de código que realiza una tarea específica sin devolver ningún valor, como imprimir un mensaje en la consola, mover un objeto gráfico o modificar una variable global. Por otro lado, una función es una estructura de código que realiza una tarea específica y devuelve un valor o conjunto de valores como resultado. Las funciones se utilizan para reutilizar código y simplificar el diseño de programas, mientras que las acciones se utilizan para realizar tareas que no requieren la devolución de resultados. Es importante conocer la diferencia entre ambas para utilizarlas correctamente en el desarrollo de programas. Las áreas de aplicación de estas estructuras son diversas y se utilizan en diferentes tipos de proyectos de programación.

Acoplamiento: relación entre módulos. Un módulo con un acoplamiento bajo debería tener una independencia adecuada de los otros módulos del programa.

Ámbito de un identificador: El ámbito de un identificador se refiere al alcance en el que una variable, función u otro objeto puede ser referenciado o utilizado dentro del código. En Python, las reglas de ámbito determinan si una variable es global o local, según su ubicación dentro del código. Las variables globales se definen fuera de cualquier función o clase y se pueden acceder desde cualquier parte del código, mientras que las variables locales se definen dentro de una función o método y solo se pueden acceder desde dentro de esa función o método. Las reglas de ámbito son importantes para evitar errores y hacer que el código sea más fácil de entender.

Áreas de aplicación: algoritmos de búsqueda y ordenación, cálculo de números factoriales, árboles binarios, criptografía, teoría de números, cálculo de series matemáticas, programación recursiva, y estructuras de datos recursivas.

Caso base: condición inicial que detiene la llamada recursiva en una función. Ejemplo: en la función para calcular el número de Fibonacci, el caso base es cuando n es igual a 0 o 1.

Caso recursivo: condición en la que la función se llama a sí misma con un argumento diferente para resolver el problema. Ejemplo: en la función para calcular el número de Fibonacci, el caso recursivo es cuando n es mayor que 1 y se llama a sí misma dos veces con los argumentos $n-1$ y $n-2$.

Cohesión: medida en que las tareas de un módulo están relacionadas entre sí. Un módulo altamente cohesivo debería contener funciones o métodos que se relacionen directamente con la funcionalidad del módulo.

Correspondencia de parámetros: proceso de asignar los valores de los parámetros actuales a los parámetros formales en una función en Python. Se utilizan dos métodos diferentes: la correspondencia por posición y la correspondencia por nombre.

Correspondencia por nombre: método de pasar los parámetros a una función en el que los valores de los parámetros actuales se asignan a los parámetros formales según su nombre. Áreas de aplicación: programación en Python.

Correspondencia por posición: método de pasar los parámetros a una función en el que los valores de los parámetros actuales se asignan a los parámetros formales según su posición. Este método de correspondencia de parámetros es el valor predeterminado en Python. Áreas de aplicación: programación en Python.

Criterios de descomposición modular: proceso clave para el diseño de programas de alta calidad y fácilmente mantenibles. Estos criterios son cohesión, acoplamiento, simplicidad y reutilización.

Efectos laterales: son cambios en el estado del programa que no están relacionados con el valor devuelto por una función en programación. En Python, los efectos laterales pueden ser sutiles y pueden conducir a errores difíciles de depurar. Ejemplo: cuando una función modifica una variable global, se está produciendo un efecto lateral en el estado del programa que no está relacionado con el valor devuelto por la función. Áreas de aplicación: Programación en general.

Función: En programación, una función es un bloque de código que realiza una tarea específica y se puede llamar desde cualquier parte del programa. Puede aceptar cero o más argumentos y devolver un valor o no devolver nada. En el texto, se menciona una función que calcula el área de un círculo a partir de su radio.

Parámetros arbitrarios: Son una forma de utilizar una lista de parámetros variables en una función en Python. Se utiliza el operador de asterisco (*) antes del nombre del parámetro formal para indicar que se trata de una lista variable de parámetros. Ejemplo: en la función "suma_arbitraria(*numeros)", "numeros" es un parámetro arbitrario que puede recibir cualquier cantidad de valores.

Parámetros formales y actuales: Los parámetros formales son variables definidas dentro de una función en Python y se utilizan para realizar una tarea específica. Por otro lado, los parámetros actuales son los valores que se pasan como argumentos a una función en Python cuando se llama a la misma. Es importante asegurarse de que la cantidad de parámetros actuales coincida con la cantidad de parámetros formales definidos en la función. Ejemplo: en la función "suma(a, b)", "a" y "b" son parámetros formales, mientras que los valores 2 y 3 en la llamada "suma(2, 3)" son los parámetros actuales.

Parámetros: En Python, los parámetros son valores que se pasan a una función o subprograma para que éste pueda realizar una tarea específica. Los parámetros permiten la transferencia de información entre el programa principal y los subprogramas. Pueden ser posicionales o de palabra clave y permiten la creación de funciones más versátiles y personalizadas. Las áreas de aplicación de los parámetros en Python son en la definición de funciones y subprogramas que requieren datos de entrada específicos para su funcionamiento.

Paso de parámetros por referencia: método de pasar los parámetros a una función en el que se pasa la ubicación de memoria de un objeto como parámetro. En este caso, cualquier cambio realizado en el objeto dentro de la función se reflejará en el objeto original fuera de la función, ya que ambas variables apuntan a la misma ubicación de memoria. Áreas de aplicación: programación en Python.

Paso de parámetros por valor: método de pasar los parámetros a una función en el que se copia el valor de un parámetro en una variable temporal dentro de la función. Cualquier cambio realizado en la variable dentro de la función no afectará el valor original fuera de la función. Esto se debe a que la variable original fuera de la función está en una ubicación de memoria diferente a la variable temporal dentro de la función. Áreas de aplicación: programación en Python.

Programación modular: técnica de diseño de software que permite dividir un programa en partes más pequeñas y manejables

Recursividad directa: tipo de recursividad en el que la función se llama a sí misma directamente. Se utiliza para resolver problemas que se pueden dividir en subproblemas similares. Por ejemplo, la función factorial se puede definir de manera recursiva.

Recursividad indirecta: tipo de recursividad en el que una función llama a otra función que, a su vez, llama a la primera función. Se utiliza cuando el problema se puede dividir en subproblemas que se resuelven con funciones diferentes pero relacionadas. Por ejemplo, la función gcd se puede definir de manera recursiva utilizando la función mod.

Recursividad mutua: tipo de recursividad en el que dos o más funciones se llaman entre sí para resolver un problema. Se utiliza cuando el problema se puede dividir en subproblemas que se resuelven con funciones diferentes pero que necesitan comunicarse entre sí. Por ejemplo, la función is_even y is_odd se pueden definir de manera recursiva.

Recursividad: La recursividad es una técnica de programación que permite que una función se llame a sí misma para resolver un problema. La recursividad se utiliza con frecuencia para resolver problemas que pueden ser expresados en términos de una serie de pasos similares. En el texto, se mencionan las ventajas de la recursividad y se proporciona un ejemplo de una función recursiva en Python que calcula el factorial de un número.

Recursividad: técnica de programación en la que una función se llama a sí misma para resolver un problema de manera iterativa. Se utiliza para dividir problemas

complejos en subproblemas más pequeños y solucionarlos por etapas. Un ejemplo de recursividad en Python es la función para calcular el número de Fibonacci.

Reutilización: un módulo bien diseñado debería poder ser reutilizado en diferentes partes del programa, y en otros programas. Esto significa que los módulos deben ser lo más genéricos posible, y no incluir referencias específicas a otras partes del programa o del entorno.

Simplicidad: los módulos en Python deben ser lo más simples y fáciles de entender posible. Esto implica que cada módulo debe realizar tareas específicas y claras, sin incluir complejidades adicionales o innecesarias.

Subprograma: En Python, un subprograma es una sección de código que realiza una tarea específica y puede ser llamada desde cualquier parte del programa principal. También se le conoce como función o procedimiento, y su uso permite la modularización del código, mejorando la legibilidad y la eficiencia del programa. Las áreas de aplicación de los subprogramas en Python son en la creación de funciones, procedimientos y métodos que realizan tareas específicas en un programa.

Subprogramas: Los subprogramas en programación se refieren a bloques de código que realizan tareas específicas y se pueden llamar desde diferentes partes del programa. Los subprogramas se utilizan para reutilizar el código, mejorar la legibilidad y estructura del código, permitir el encapsulamiento del código y mejorar la eficiencia del programa. En el texto, se mencionan las ventajas de los subprogramas y cómo se utilizan en Python.

Variables formales: son parámetros definidos en una función que se utilizan para recibir valores desde la llamada a la función en Python. Su alcance está limitado a la función en la que se define, pero su valor es definido por la llamada a la función.

Ejemplo: en la función "restar(numero1, numero2)", los parámetros "numero1" y "numero2" son variables formales que se utilizan para recibir valores desde la llamada a la función. Áreas de aplicación: Programación en Python.

Variables locales: son variables definidas dentro de una función en Python, su alcance está limitado a la función en la que se define y no se puede acceder desde fuera de ella. Ejemplo: en la función "sumar(numero1, numero2)", la variable local

"resultado" solo es accesible dentro de la función "sumar". Áreas de aplicación:
Programación en Python.